Data Structures

Assignment 2: ImageLab

Total marks	100 (Phase 1: 50, Phase 2: 50)	**Submission**	Google Classroom (GCR)
Phase 1 due	16/10/2026	**Phase 2 due**	16/10/2026

1. Important Guidelines

1. Submit each phase on time. Late submissions will not be accepted and late queries will not be entertained.
2. You are responsible for submitting working files and the correct code. A corrupted or wrong submission receives 0 marks.
3. The policy on plagiarism and AI use is strict. No exceptions will be made.
4. GCR is the only channel for assignment queries. Emails about the assignment will not be answered.
5. Marks are awarded only for behaviour that is **demonstrated at run time**: in the program output, the saved images, the Data Structure Inspector, or the GUI if you build one. A concept that exists in the code but cannot be shown to the TA earns no marks.
6. If you cannot explain your code during the demo, marks will be deducted accordingly.
7. Follow the folder structure and naming scheme in Section 9. Not following it costs 20% of the phase marks.
8. Please Note that the deadline for phase 1 and phase 2 is same

2. Overview

Modern applications use fundamental data structures far beyond textbook examples. Image editors, browsers, task schedulers, operating systems and compilers rely on stacks, queues and priority queues for history management, traversal, buffering and scheduling.

In this assignment you will develop **ImageLab**, a miniature image-processing application whose internal design is deliberately driven by data structures. You will implement the stack, circular queue and priority queue from scratch and use them to power undo/redo, memory-aware history, region filling and job scheduling.

The assignment has two phases. **Phase 1** builds the data structures and the core editor. **Phase 2** extends the same code with a memory-aware history, a priority scheduler, experiments and analysis. Phase 2 builds on your Phase 1 code; you may fix Phase 1 problems in Phase 2, but Phase 1 will not be re-marked.

3. Rules for Libraries and Language Features

Allowed	Not allowed (0 marks for the affected part)
Raw arrays, dynamic arrays (`new` / `delete`), pointers, structs, classes and templates `std::string`, `<iostream>`, `<fstream>`, `<cmath>`, `<cstdlib>`, `<chrono>` or `<ctime>` for timing An image library **only** to read an image file into a pixel array and write a pixel array to a file A GUI library **only** to show windows, images and controls (GUI is optional, see Section 4)	Any standard container: `std::vector`, `std::list`, `std::deque`, `std::stack`, `std::queue`, `std::priority_queue`, `std::map`, `std::set`, `std::array` and similar Library functions that do the image operation for you: rotate, flip, resize, crop, blur, brightness, colour conversion, flood fill **Recursion** for the bucket fill (you must use your own stack and queue) Threads, mutexes or any concurrency library

4. Application Scenario and User Interface

ImageLab lets a user load an image, apply edits and filters, undo and redo them, queue slower processing jobs, fill connected regions, and inspect the internal state of the data structures.

- A **console (menu-driven) program** is accepted for full marks. Regions and pixels are entered as coordinates, and results are written to image files.
- A **GUI is optional** and earns bonus credit (Section 10). It may be simple, but it must not replace any required data-structure work.
- In both cases the program must clearly show the data-structure behaviour through the Data Structure Inspector (Section 6.7).

5. Required Class Interfaces

Use the class and function names from the lectures (Malik). Every class must be a **template**, and must manage its own memory (destructor, copy constructor and overloaded assignment operator). The TAs will test your classes through these functions, so do not rename them. You may add private members and extra public functions.

```
template <class Type>
class stackType                       // array or linked list: your choice
{
public:
    void initializeStack();           // make the stack empty
    bool isEmptyStack() const;
    bool isFullStack() const;         // linked version always returns false
    void push(const Type& newItem);
    Type top() const;                 // stack must not be empty
    void pop();                       // stack must not be empty
    int  size() const;                // number of elements
    stackType(int stackSize = 100);
};

template <class Type>
class queueType                       // CIRCULAR array; no shifting of elements
{
public:
    void initializeQueue();           // make the queue empty
    bool isEmptyQueue() const;
    bool isFullQueue() const;
    Type front() const;               // queue must not be empty
    Type back() const;                // queue must not be empty
    void addQueue(const Type& queueElement);
    void deleteQueue();               // queue must not be empty
    int  size() const;                // number of elements
    int  capacity() const;            // maximum number of elements
    queueType(int queueSize = 100);
};

template <class Type>
class priorityQueueType               // Phase 2: array or linked list
{
public:
    void initializeQueue();
    bool isEmptyQueue() const;
    bool isFullQueue() const;
    void addQueue(const Type& queueElement);   // insert according to priority
    Type front() const;                        // highest-priority element
    void deleteQueue();                        // remove the highest-priority element
    int  size() const;
};
```

- **Empty and full cases** (pop or top on an empty stack, deleteQueue on an empty queue, addQueue on a full queue) must never crash the program. Handle them as in the lectures and state your choice in the report.
- To **display** the contents of a structure (for the Inspector or the pending-job list) you may add a read-only function that visits the elements without removing them.
- The bucket fill needs large capacities. Either create the structure with enough capacity or let an array-based structure grow by allocating a larger array. A growing circular queue must keep its elements in the correct order.

6. Phase 1: Data Structures and Core Editor (50 marks)

6.1 Core Image Operations

Implement every operation yourself on the pixel array. Operations marked **instant** run immediately; operations marked **queued** are submitted to the processing queue (Section 6.5).

Operation	Type	Input	Expected output
Load image	Instant	File name	Image loaded into memory
Crop	Instant	Rectangular region	Image containing only the selected region
Rotate	Instant	Angle: 90°, 180° or 270°	Image rotated by the angle
Flip	Instant	Direction: horizontal or vertical	Image mirrored along the chosen axis
Brightness	Instant	Positive or negative adjustment value	Brightness increased or decreased; values stay within 0 to 255
Grayscale	Instant	Colour image	Corresponding grayscale image
Resize	Queued	Target width and height	Image resized to the given dimensions
Blur	Queued	Blur strength or kernel size	Neighbouring pixel variations smoothed
Bucket fill	Queued	Starting pixel (x, y) and replacement colour	Connected region of the starting pixel changed to the replacement colour (Section 6.6)
Save project	Queued	None	Current image written to the project's working file
Export	Queued	New file name	Current image written to a new file chosen by the user

6.2 Custom Stack

Implement `stackType` as specified in Section 5, using either an array (fixed or growing) or a linked list. Your report must justify the representation you chose and give the complexity of each operation.

6.3 Undo and Redo

Every editing operation that changes the image must be recorded as an **editing command** (a struct or class holding the operation type, its parameters, and the information needed to reverse it). Maintain two of your stacks: an **Undo stack** and a **Redo stack**.

- Undo moves the most recent command from the Undo stack to the Redo stack and reverses it.
- Redo moves the command back and applies it again.
- A new operation performed after one or more undos **clears the Redo stack**.
- Queued operations are recorded in the history when they are executed, not when they are submitted.

- In Phase 1 a command may store a full copy of the previous image. Phase 2 replaces this with compact records (Section 7.1).

```
After:  Rotate -> Crop -> Blur
Undo stack (top first):  Blur, Crop, Rotate        Redo stack:  (empty)
Undo once:
Undo stack (top first):  Crop, Rotate              Redo stack:  Blur
```

6.4 Custom Circular Queue

Implement `queueType` as specified in Section 5. It must be a **circular** array queue: an implementation that shifts the remaining elements after each deleteQueue is not acceptable. State in the report how you tell an empty queue from a full one (count variable or reserved slot).

6.5 Processing Pipeline

Queued operations are placed in a processing queue instead of running immediately. Jobs are processed in **FIFO** order.

```
FRONT -> [Resize] [Blur] [Bucket fill] [Blur] [Export] -> REAR
```

The user must be able to:

- **Add** a job, and **list** the pending jobs in order without removing them.
- **Process** the next job, or process all pending jobs.
- **Cancel** a pending job. Do not shift the queue: mark the job as cancelled and skip it when it reaches the front.
- **Clear** all pending jobs.

Bounded queue. The queue has a configurable maximum capacity. When it is full the program must behave gracefully: reject the new job, ask the user to process pending jobs first, or apply another defensible policy. Explain your policy in the report.

Undo and Redo while jobs are pending. If no jobs are pending, Undo and Redo run immediately. If jobs are pending, the request is added to the queue as a job, so that it acts on the image at the correct point in the sequence.

6.6 Bucket Fill with a Stack and with a Queue

Bucket fill behaves like the paint-bucket tool in common image editors. The user selects a starting pixel (x, y) and a replacement colour. All pixels in the **connected region** of the starting pixel that have the starting pixel's original colour are changed to the replacement colour.

Implement the traversal **twice**, and let the user choose which one runs:

Version	Data structure	Flow
Stack-based fill	Your `stackType`	push start → pop → process → push valid neighbours
Queue-based fill	Your `queueType`	addQueue start → deleteQueue → process → addQueue valid neighbours

- Only the **up, down, left and right** neighbours are connected. Diagonal pixels are not.
- Only the region containing the starting pixel changes. Other regions of the same colour stay unchanged.
- Never access a pixel outside the image.
- If the replacement colour equals the original colour, the operation must finish without looping forever.
- **Recursion is not allowed.** A recursive fill uses the system stack instead of your stack and overflows on large images.
- Both versions must produce exactly the same image for the same input.

The two versions visit the pixels in different orders. The stack-based fill follows one direction as far as it can before coming back; the queue-based fill spreads outward from the starting pixel layer by layer. You will compare them in Phase 2 (Section 7.5).

6.7 Data Structure Inspector

ImageLab must include a Data Structure Inspector that shows the internal state of your structures. In a console program it is a menu option that prints the values; in a GUI it may be a panel or dialog.

Metric	Phase 1	Phase 2
Undo stack size and Redo stack size	Required	Required
Current queue size and maximum recorded queue size	Required	Required
Number of push and pop operations	Required	Required
Number of addQueue and deleteQueue operations	Required	Required
Contents of the Undo stack, Redo stack and pending jobs (operation names)	Required	Required
History memory consumption and memory budget	Not required	Required
Average queue waiting time	Not required	Recommended

6.8 Phase 1 Testing and Deliverables

Test the data structures on their own before using them in ImageLab. Submit a separate test program that prints PASS or FAIL for each case.

Component	Minimum cases
Stack	Pop and top on an empty stack; large number of pushes; repeated push and pop; initializeStack; copy constructor and assignment
Circular queue	deleteQueue on an empty queue; full queue; wrap-around; repeated addQueue and deleteQueue; overflow and underflow handling
Undo / Redo	Several undos; several redos; new action after undo clears the Redo stack
Bucket fill	Single-pixel region; whole image; irregular region; region touching the image border; several separate regions of the same colour; stack-based and queue-based fills give identical output

Phase 1 report (at most 3 pages): representation chosen for the stack and the queue with justification; complexity of each operation with a one-line reason; your full-queue policy; test results.

7. Phase 2: History, Scheduling and Evaluation (50 marks)

7.1 Memory-Aware History

The history (Undo and Redo stacks together) must work within a **configurable memory budget**, for example 50 MB. Use a small budget such as 5 MB when testing so that the limit is actually reached.

Each history record must hold: operation type, parameters, time or sequence number, estimated memory used by the record, affected region, and enough information to reverse the operation.

Storing a full copy of the image for every operation loses marks. Store only what is needed to reverse each operation:

Operation	What can be stored to reverse it
Rotate, Flip	Nothing but the parameters: the inverse is another rotate or flip

Brightness	The adjustment value, plus the original values of any pixels that were clipped at 0 or 255 (clipping loses information)
Crop	The removed pixels and the crop coordinates
Bucket fill	The original colour and the set of pixels that changed
Grayscale, Blur, Resize	These cannot be reversed from parameters. Store the previous pixels of the affected region

Cleanup. When the budget is exceeded the system must automatically discard history records. The usual policy is to discard the **oldest** records first. Removing the oldest record is not a normal stack operation, so for the history only you may add a function such as `removeBottom()` to your stack. Document and justify your cleanup policy.

7.2 Command Coalescing

Consecutive compatible operations must be combined into one history record. Coalescing brightness adjustments is required; other compatible operations (for example two rotations) are optional.

```
Brightness +1, +1, +1, +1            ->  one record: Brightness +4
Brightness +2, Crop, Brightness +3   ->  three records (order must be preserved)
```

- Only records that are next to each other on the Undo stack may be combined.
- Undoing a combined record reverses all of its operations in one step.
- Explain in the report how your design stays correct when pixel values are clipped (for example +200 followed by −100 is not the same as +100).

7.3 Priority Queue and Scheduling

Interactive applications contain tasks of different urgency: a quick request should not have to wait behind a long export. Implement `priorityQueueType` (Section 5) using an **ordered or unordered array or linked list**, as covered in the lectures. A heap is not expected.

Each job must hold at least: job ID, job type, priority, arrival time or sequence number, and an estimated or measured cost. A smaller number means a higher priority.

Job	Priority
Undo / Redo request (submitted while jobs are pending)	1 (highest)
Blur	2
Bucket fill	2
Resize	3
Save project	4 (finalization)
Export	4 (finalization)

ImageLab must offer two scheduling modes: **FIFO** (your `queueType` from Phase 1) and **Priority** (your `priorityQueueType`).

- **Ties.** Jobs with the same priority are served in arrival order.
- **Finalization rule.** Save and Export are barriers. A Save or Export job runs only after every job submitted before it has finished, and no job submitted after it may run before it. Priority order therefore applies only among the jobs between two finalization jobs.

```
Submitted:        Resize(3)  Blur(2)  Export(4)  Blur(2)  Undo(1)
Priority order:   Blur  Resize  Export  Undo  Blur
                  (the last two jobs cannot overtake Export)
```

Comparison. Create one workload of at least 30 jobs with a mix of all job types and run it to completion under both modes. Report the average waiting time, average completion time, average waiting time of priority-1 jobs, and number of jobs processed. Discuss when FIFO is preferable and when priority scheduling improves responsiveness. Waiting time is the time from submission until the job starts; use measured execution times or your estimated costs, and say which.

7.4 Batched Operations

Extend your stack and circular queue with batch operations:

```
void pushBatch(const Type items[], int n);          void popBatch(int n);
void addQueueBatch(const Type items[], int n);      void deleteQueueBatch(int n);
```

A batch operation must do better than calling the single operation n times (for example one capacity check, one resize, or copying a block at a time across the wrap-around). Compare single and batched operations for 1,000, 10,000 and 100,000 items, report the difference and explain it.

7.5 Experimental Evaluation

Implementation alone is not sufficient. Run controlled experiments and present the results in tables or graphs. Repeat each timing at least three times and report the average.

Experiment	Workloads	Metrics
Stack-based vs queue-based bucket fill	Image sizes 512×512, 1024×1024, 2048×2048 (4096×4096 if your machine allows)	Execution time; number of insertions; maximum stack or queue size; order in which pixels are visited
History	100 and 1,000 operations under a fixed budget	Memory used; records discarded; undo time; effect of coalescing on record count
FIFO vs Priority	The workload from Section 7.3	Metrics listed in Section 7.3
Single vs batched	1,000, 10,000 and 100,000 items	Execution time

7.6 Complexity Analysis

Give the time and space complexity of the major operations. Do not only list Big-O values: explain in one or two lines why each complexity occurs and how your implementation affects it.

Operation	Required analysis
Stack push, pop, top	Time and space
Queue addQueue, deleteQueue, front	Time, and how the circular array achieves it
Priority queue addQueue, deleteQueue	Time for your chosen representation (ordered or unordered)
Stack-based and queue-based bucket fill	Time and auxiliary space
Undo and Redo	Time and history space
Command coalescing	Time and the conditions for merging
History cleanup	Time and memory policy
Batch operations	Time compared with n single operations

7.7 Phase 2 Testing

Extend your Phase 1 test program with the following cases.

Component	Minimum cases
Priority queue	Same-priority jobs; mixed priorities; large workload; tie handling; finalization rule

Memory-aware history	Budget reached; oldest records discarded; undo after cleanup; undo of each operation type restores the exact previous image
Coalescing	Repeated brightness; brightness separated by another operation; undo and redo of a combined record
Batch operations	Batch larger than free space; batch across the wrap-around; popBatch or deleteQueueBatch with n larger than size

7.8 Phase 2 Report

Submit a report of at most 10 pages with these sections:

1. Design of the history records and the cleanup policy, with justification
2. Coalescing rules and how correctness is preserved
3. Scheduler design: priority queue representation, tie handling and the finalization rule
4. Experimental results (Section 7.5) with tables or graphs and a short discussion of each
5. Complexity analysis (Section 7.6)
6. Testing evidence (Sections 6.8 and 7.7)
7. Known limitations

8. Marks Distribution

Phase 1 component	Section	Marks
Core image operations implemented by hand	6.1	8
Stack (`stackType`)	6.2	7
Undo and Redo with two stacks	6.3	9
Circular queue (`queueType`)	6.4	7
Processing pipeline: add, list, process, cancel, clear, bounded queue	6.5	6
Bucket fill: stack-based and queue-based versions	6.6	8
Data Structure Inspector, tests and Phase 1 report	6.7, 6.8	5
Phase 1 total		**50**

Phase 2 component	Section	Marks
Memory-aware history: compact records, budget, cleanup	7.1	10
Command coalescing	7.2	5
Priority queue and scheduling, including the finalization rule	7.3	9
Batched operations	7.4	4
Experimental evaluation	7.5	8
Complexity analysis	7.6	5
Testing evidence	7.7	4
Report quality and explanation in the demo	7.8	5
Phase 2 total		**50**

How each component is graded

Level	Description

Full marks	Works correctly for normal, boundary and error cases; demonstrated at run time; student explains the design and the code
Partial marks	Works for normal cases but fails some boundary or error cases, or the student can only partly explain it
No marks	Not demonstrated at run time, uses a forbidden library feature, or the student cannot explain it

9. Submission

Submit one zip file per phase on GCR. Name the main folder with your roll number, section, assignment and phase, for example `25i-0000_A_A2_P1` and `25i-0000_A_A2_P2`.

```
25i-0000_A_A2_P1/
    src/         all source and header files
    tests/       the test program for the data structures
    images/      the small test images you used
    report/      the report as a PDF
    README.txt   how to build and run, and which libraries are needed
```

Checklist before you submit

- The project builds and runs from a clean copy using the steps in README.txt
- No standard container and no forbidden library function is used anywhere
- Class and function names match Section 5
- The test program runs and prints PASS or FAIL for every required case
- The Inspector shows every required metric for the phase
- The report is a PDF within the page limit
- The folder name and structure follow this section

10. Bonus (up to 10 marks)

Bonus credit is considered only after all mandatory requirements of both phases are complete. It does not compensate for missing mandatory work. Examples:

- A working graphical user interface (up to 5 marks)
- Graphical animation of the stack and queue contents
- Side-by-side visualization of the stack-based and queue-based fills
- Layer-management system
- Persistent project history or checkpoints
- Adaptive queue capacity
- More sophisticated command compression
- Detailed performance dashboard

HAPPY CODING